

Task 5: Understanding the useState Hook

Summary

In this task, I learned about the useState Hook, which is one of the most important Hooks in React. It allows functional components to store and manage state, which represents data that can change during the execution of an application. Whenever the state changes, React automatically re-renders the component so that the user interface stays updated.

The useState Hook is imported from React and is declared at the top level of a functional component. It returns an array containing two values: the current state variable and a setter function that is used to update that state.

```
const [count, setCount] = useState(0);
```

I learned that state can store different types of values, such as numbers, strings, booleans, arrays, and objects. It is commonly used for features like counters, form inputs, checkboxes, and dynamic user data. To change the state, the setter function returned by useState is used instead of modifying the value directly.

Another important concept I learned is that state updates are asynchronous. Calling the setter function does not immediately update the state value in the current execution. Instead, React schedules the update and applies it during the next render. When multiple updates depend on the previous state, using an updater function (`prevState => prevState + 1`) ensures that the latest state value is used correctly.

I also understood how to manage objects and arrays in state. Instead of directly modifying existing objects or arrays, React recommends creating a new copy using the spread operator (...). This helps React detect changes efficiently and update the UI correctly.

Additionally, I learned that the initial state is created only once during the first render. If the initialization process is expensive, an initializer function can be passed to useState to improve performance. I also explored common mistakes such as mutating state directly, logging state immediately after updating it, and causing infinite re-renders by updating state during rendering.

Learning Outcome

After completing this task, I understood how the useState Hook works, how to create and update state in functional components, manage different data types safely, update state based on previous values, and avoid common mistakes while building dynamic and interactive React applications.

You pressed me 0 times

hello

You typed: hello

Reset

☒ I liked this

You liked this.

Taylor

Increment age

Hello, Taylor. You are 42.

With updater function:

Your age: 42

+3

+1

Passing the next state directly:

Passing the next state directly:

Your age: 42

+3

+1

Add todo

Add

☒ Buy milk

Edit

Delete

☐ Eat tacos

Edit

Delete

☐ Brew tea

Edit

Delete

Add

• Item 1

• Item 2

• Item 3

• Item 4

• Item 5

• Item 6

• Item 7

• Item 8

• Item 9

• Item 10

• Item 11

• Item 12

• Item 13

• Item 14

• Item 15

• Item 16

• Item 17

Task 6: Responding to Events in React

Summary

In this task, I learned how React handles user interactions using event handlers. Event handlers are JavaScript functions that execute when users perform actions such as clicking a button, typing in an input field, hovering over an element, or submitting a form. React uses camelCase event names such as `onClick`, `onChange`, and `onSubmit` to handle these events.

To respond to an event, a function is first defined inside the component and then passed as a prop to the appropriate JSX element. The correct way to attach an event handler is by passing the function reference, such as `onClick={handleClick}`. Calling the function directly using `onClick={handleClick()}` is incorrect because it executes immediately during rendering instead of waiting for the user interaction.

I also learned that event handlers can be written in different ways, including separate functions, inline functions, and arrow functions. Since event handlers are declared inside components, they can easily access the component's props and state. Parent components can also pass event handler functions to child components through props, allowing different components to communicate while keeping the code reusable and organized.

Another important concept is event propagation (event bubbling). When an event occurs on a child element, it automatically travels upward to its parent elements. In situations where this behavior is not required, React provides the `event.stopPropagation()` method to stop the event from reaching parent components.

I also learned about preventing the browser's default behavior using `event.preventDefault()`. This is especially useful when working with forms because it prevents the browser from reloading the page after form submission, allowing React to handle the event completely.

Finally, I understood that event handlers are the appropriate place to perform actions such as updating state, displaying messages, handling user input, and responding to user interactions. They make React applications dynamic, interactive, and responsive while maintaining clean and reusable component design.

Learning Outcome

After completing this task, I understood how React handles user events, how to write and attach event handlers correctly, how to pass event handler functions between components, and how to control event propagation and default browser behavior using `stopPropagation()` and `preventDefault()`. These concepts are essential for building interactive React applications.

Click me	
Play Movie	Upload Image
Play 'Kiki's Delivery Service'	Upload Image
Play Movie	Upload Image
Play Movie	Upload Image
	Send

⇒ **GITHUB REPO LINK**

https://github.com/yashrathod910/15D_Internship/tree/main/DAY4